

```

def __eq__(self, other): # eq is "Equal"
❸    return self._comparisonOperatorHelper(operator.eq, other)

def __ne__(self, other): # ne is "Not Equal"
❹    return self._comparisonOperatorHelper(operator.ne, other)

def __lt__(self, other): # lt is "Less Than"
❺    return self._comparisonOperatorHelper(operator.lt, other)

def __le__(self, other): # le is "Less than or Equal"
❻    return self._comparisonOperatorHelper(operator.le, other)

def __gt__(self, other): # gt is "Greater Than"
❼    return self._comparisonOperatorHelper(operator.gt, other)

def __ge__(self, other): # ge is "Greater than or Equal"
❽    return self._comparisonOperatorHelper(operator.ge, other)

```

Our comparison dunder methods call the `_comparisonOperatorHelper()` method ❶ and pass the appropriate function from the `operator` module for the `operatorFunc` parameter. When we call `operatorFunc()`, we're calling the function that was passed for the `operatorFunc` parameter—`eq()` ❸, `ne()` ❹, `lt()` ❺, `le()` ❻, `gt()` ❼, or `ge()` ❽—from the `operator` module. Otherwise, we'd have to duplicate the code in `_comparisonOperatorHelper()` in each of our six comparison dunder methods.

NOTE

Functions (or methods) like `_comparisonOperatorHelper()` that accept other functions as arguments are called higher-order functions.

Our `WizCoin` objects can now be compared with other `WizCoin` objects ❷, integers and floats ❸, and sequence values of three number values that represent the galleons, sickles, and knuts ❹. Enter the following into the interactive shell to see this in action:

```

>>> import wizcoin
>>> purse = wizcoin.WizCoin(2, 5, 10) # Create a WizCoin object.
>>> tipJar = wizcoin.WizCoin(0, 0, 37) # Create another WizCoin object.
>>> purse.total, tipJar.total # Examine the values in knuts.
(1141, 37)

```